

Progress

AI-RAN GPU Isolation — Mechanism Deep-Dive

Changjong Kim

Bigdata and HPC Lab

Department of Computer Science and Engineering

Seoul National University of Science and Technology

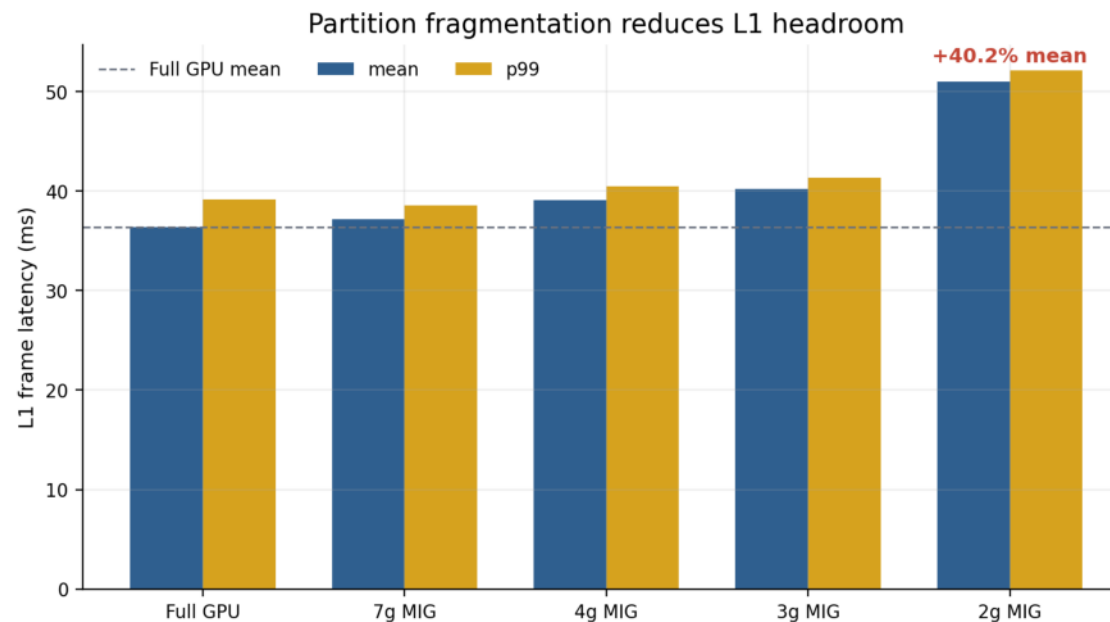
2026-06-20

Recap previous results, then drill into the mechanism.

Recap & Today's Question

From overhead measurement to mechanism identification

- Previous (5/29): MIG single-tenant overhead **+13-32ms** (Exp1-6)
- Today: AI co-tenant 환경에서 비용은 어디서, 왜 발생하는가?
- Scale: **n=500-1000/condition** (vs n=20 last time) — statistical power 25×
- Platforms: CloudLab d8545 driver 550 + [Perlmutter cross-validation](#)
- Goal: identify the **hardware resource** behind the overhead

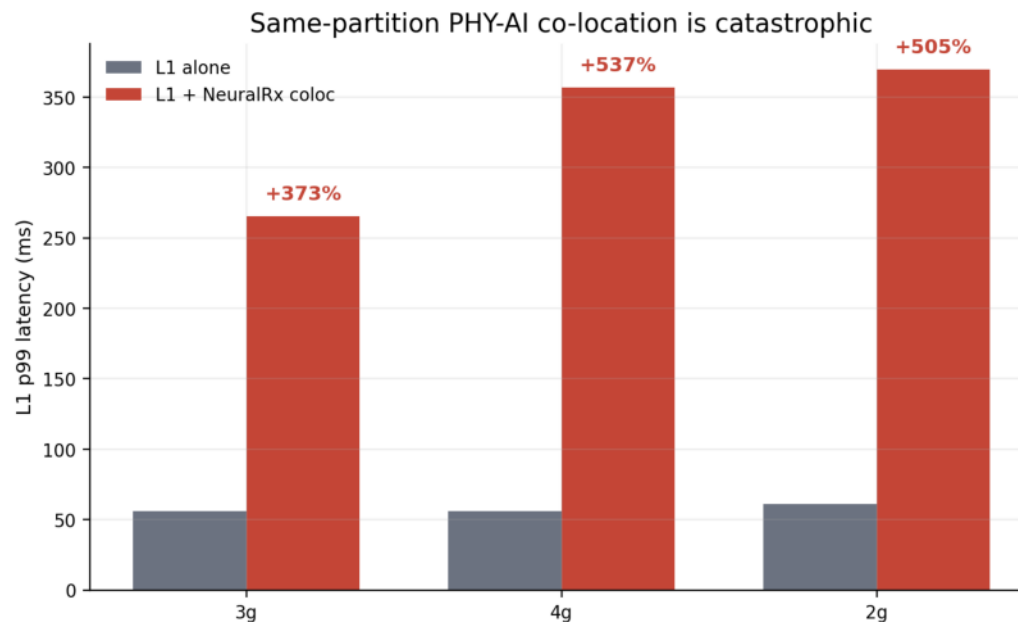


Cross-partition은 봤는데, same-partition은 어떻게 되나?

Exp7) Same-Partition Coloc = Catastrophic Collapse

Cross-partition stable, same-partition collapses 8×

- Cross-partition stays ~45ms, same-partition coloc collapses to ~357ms (8×
- 12+ workloads (chanpred, NeuralRx, qwen, xapp, ResNet) all collapse to ~357ms
- Workload-invariant → systemic mechanism, not a workload-specific trick
- n=500/condition, std=9ms (stable, reproducible)
- First evidence: cost is from placement, not the AI's nature



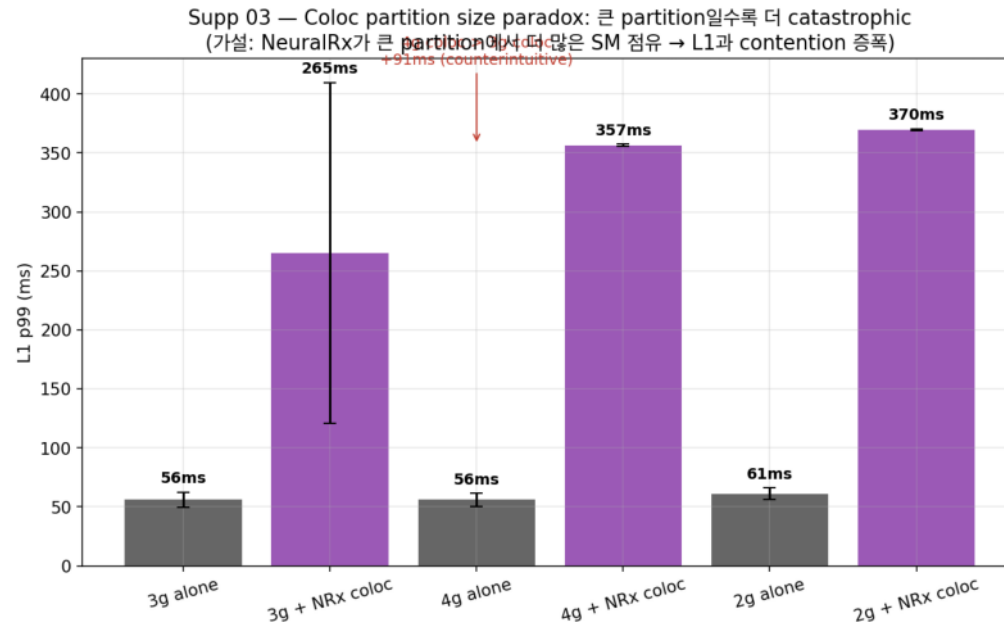
Workload	Cross-part (ms)	Same-part coloc (ms)
chanpred	45	357
NeuralRx	—	360
qwen	—	357
xapp	—	358
ResNet	—	358

큰 partition으로 SM/HBM을 더 주면 해결되나?

Exp8) Larger Partition = MORE Catastrophic

Counter-intuitive partition size paradox

- Larger slice does NOT help; 4g coloc (371ms) WORSE than 2g (369ms)
- Resource bandwidth share is decisive — but NOT via partition allocation
- Rejects the obvious 'give more SMs' mitigation hypothesis
- Bottleneck is outside the partition boundary
- First hint: the costly resource is chip-global, not slice-local



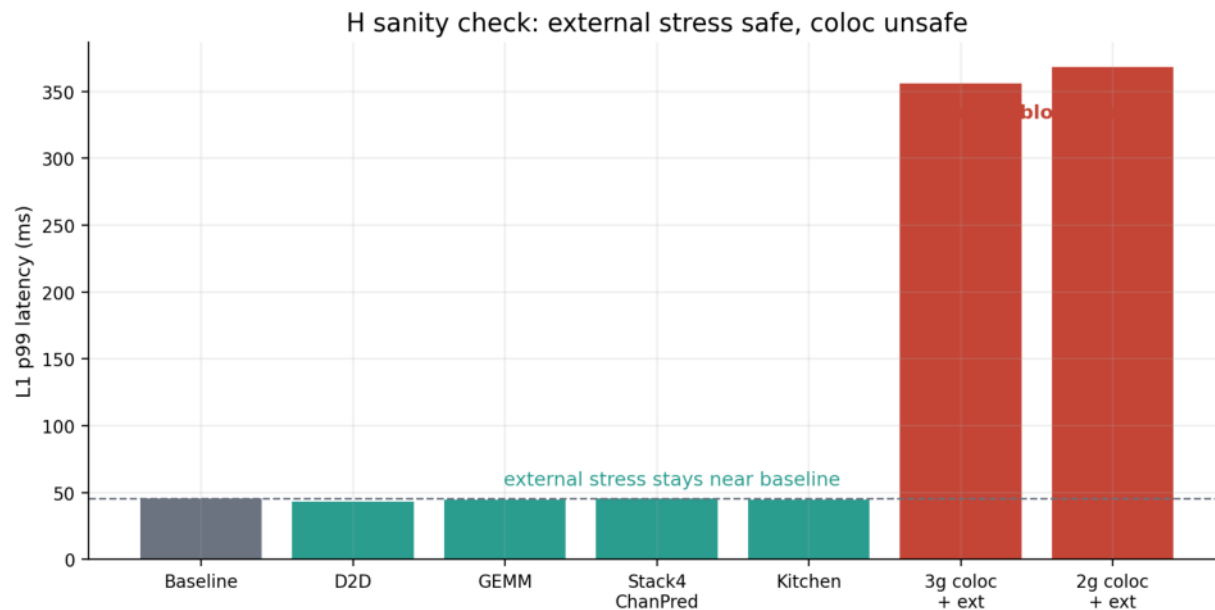
Partition	Coloc latency (ms)
2g.10gb	369
3g.20gb	361
4g.20gb	371

Placement는 binary인가, gradient인가?

Exp9) Placement = Binary Decision, No Soft Transition

One config bit decides 8× collapse

- Same workload (chanpred), only placement differs: cross 44ms ↔ coloc 357ms
- No middle ground, no gradient between the two regimes
- One config mistake = 8× collapse — operational fragility
- Implication: there exists a single shared resource flipping state
- Now we must find WHICH resource — switch to NSYS profiling



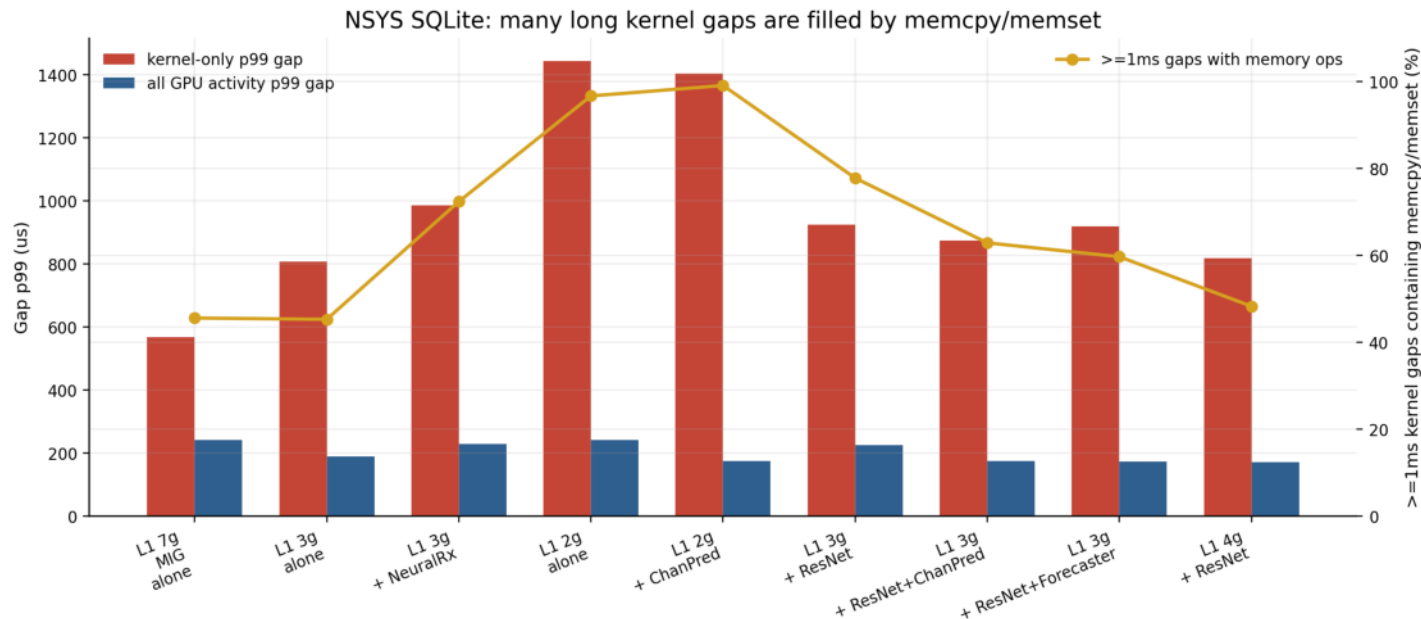
Condition	Placement	Latency (ms)
H_F1	cross-partition	44
H_G1	same-partition coloc	357

같은 hardware에서 왜 8× 차이? 무엇이 변하는가?

NSYS Layer 1 — Gap is at Malloc/Memset Boundary

Cost is NOT inside the kernel — it is between kernels

- L1 kernel duration is unchanged under coloc
- Inter-kernel 'gap' grows by exactly the missing latency
- Gap is NOT random idle — concentrated at malloc/memset boundaries
- Aggregated across 30 conditions → robust, not cherry-picked
- The bottleneck is a memory-transfer wait, not compute



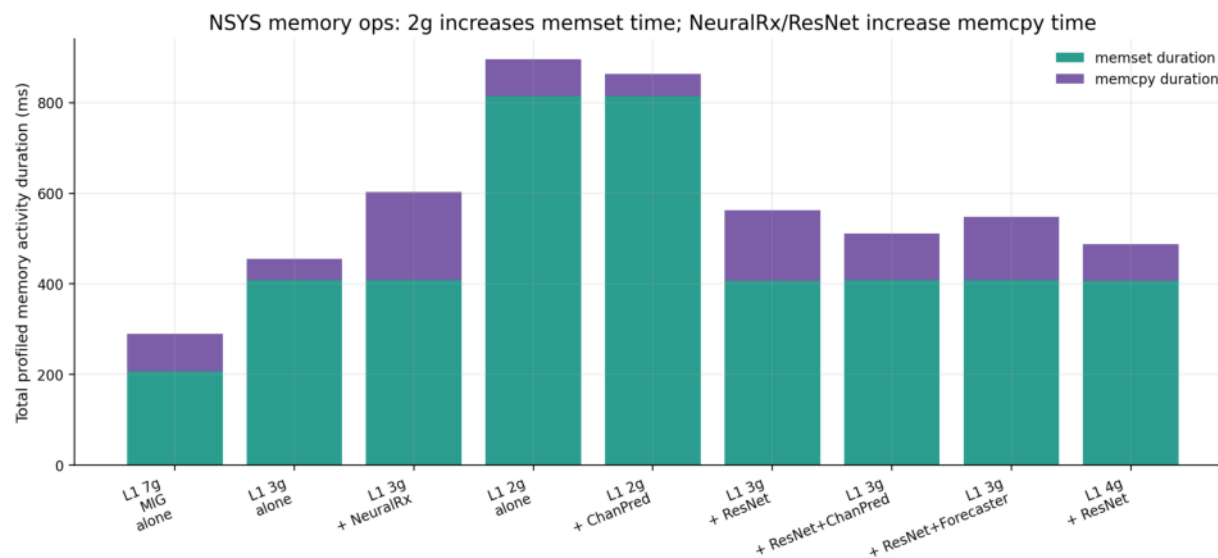
Gap이 늘었다면 — op count가 늘었나, op duration이 늘었나?

PART B §12 (CloudLab 5/31 NSYS deep, 30-condition aggregate)

NSYS Layer 2 — Same Ops, Just Slower

Rejects 'more work' hypothesis — count invariant, duration inflates

- memcpy call count: 5,778 → 5,778 (0% change)
- memset count: unchanged
- memcpy avg duration: 4.2 → 9.8μs (+133%)
- AI is NOT giving L1 more work — same work, blocked longer
- Confirms: cost is waiting, not extra activity



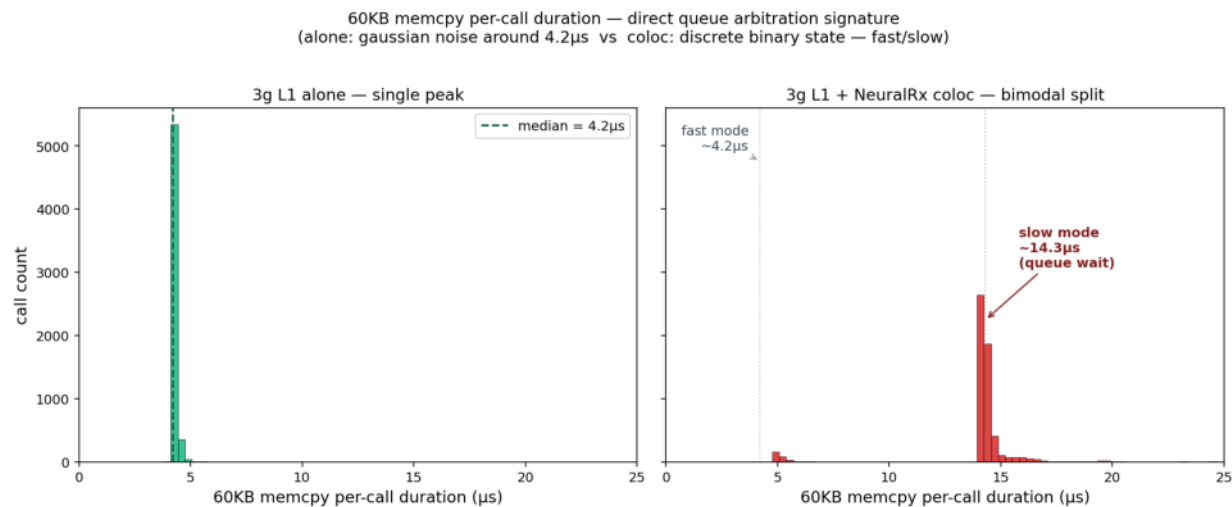
Metric	Alone	Coloc	Change
memcpy count	5,778	5,778	0%
memcpy avg μs	4.2	9.8	+133%

왜 같은 op가 더 오래 걸리나? 무엇이 op를 막는가?

NSYS Layer 3 — Bimodal Signature = Direct Queue Evidence

Per-call duration splits into two discrete modes

- Per-call 60KB memcpy duration splits into TWO discrete modes
- Alone: single gaussian peak at 4.2μs
- Coloc: bimodal 4.2μs ↔ 14.3μs (fast=queue empty, slow=queue waiting)
- Discrete state, not gaussian noise → arbitration signature
- This is the smoking gun for a shared serialized queue



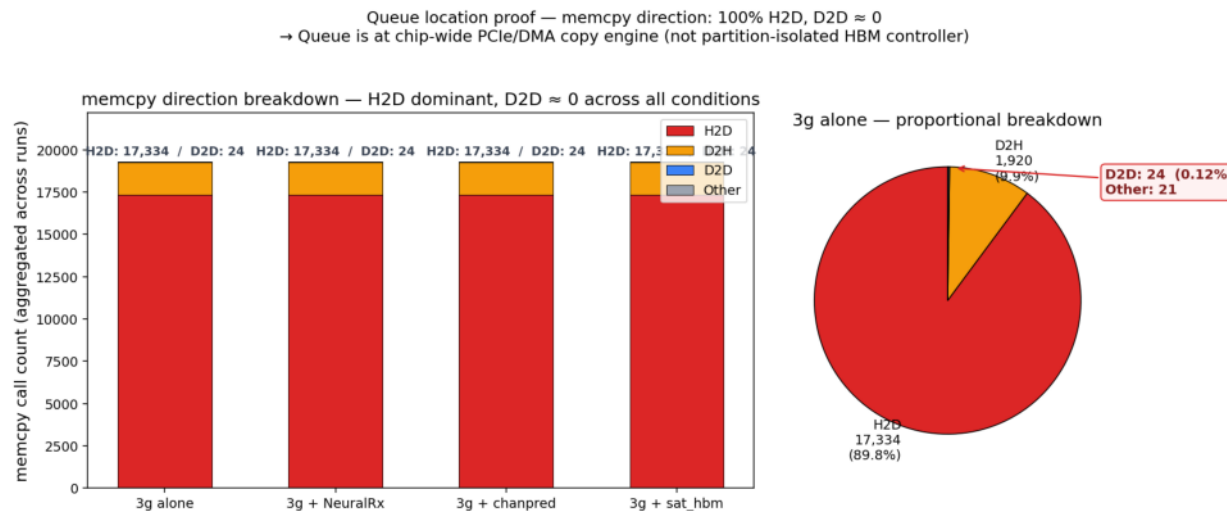
Mode	Alone (μs)	Coloc (μs)
Fast peak	4.2	4.2
Slow peak	—	14.3
Distribution	unimodal	bimodal

어떤 queue? 위치를 알아야 격리 가능성을 판단한다.

NSYS Layer 4 — Chip-Wide PCIe/DMA Copy Engine

CLIMAX — the shared resource is identified

- Direction breakdown: H2D 17,334 (89.8%) / D2H 1,920 (9.9%) / D2D 24 (0.12%)
- $D2D \approx 0$ → queue is NOT at HBM controller (HBM carries D2D)
- 100% host-path → queue is at chip-wide PCIe/DMA copy engine
- This single resource is shared across the WHOLE chip — partition cannot isolate it
- Explains: Slide 3 paradox, Slide 4 binary, Slide 7 bimodal — all consistent



Direction	Count	Share
H2D	17,334	89.8%
D2H	1,920	9.9%
D2D	24	0.12%
Other	21	0.11%

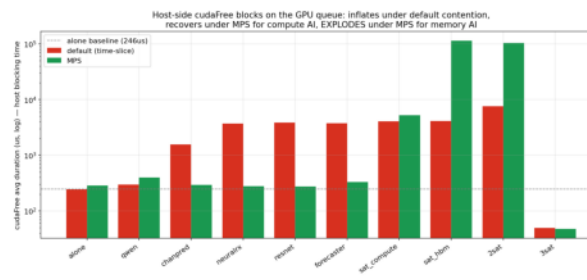
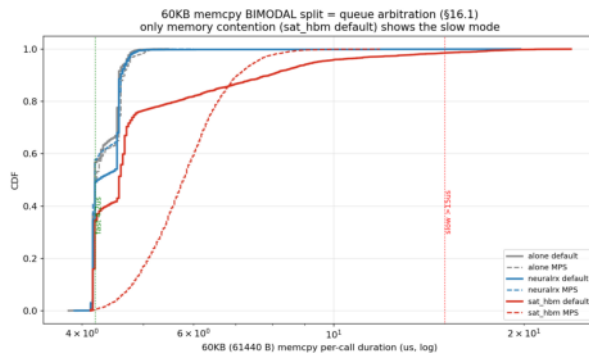
이건 MIG 결함인가, 아니면 NVIDIA stack 전체의 문제인가?

PART D §20.1, §23 (NSYS direction breakdown, 4 conditions)

Cross-Platform Validation — Mechanism is Universal

Same signature on Perlmutter (no-MIG, different driver, different cluster)

- (A) Bimodal signature reproduced: $4.2 \leftrightarrow 16.8\mu\text{s}$ on Perlmutter
- (B) cudaFree blow-up: alone $246\mu\text{s}$ → NeuralRx $3,752\mu\text{s}$ (15×) → sat_hbm MPS $115,506\mu\text{s}$ (115ms!)
- (C) correlationId causal proof: 60-82% of GPU gap overlaps EXACTLY with cudaFree host-blocking
- Conclusion: PCIe/DMA queue is an NVIDIA-stack-wide fundamental property
- Not a MIG bug — disabling MIG does not escape it



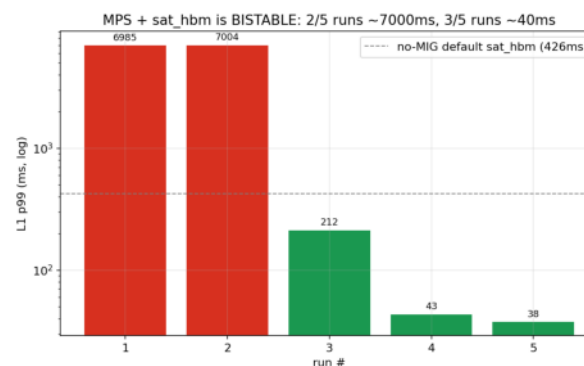
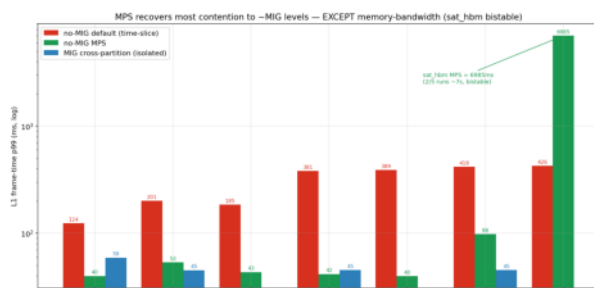
Metric	Alone	NeuralRx default	sat_hbm MPS
cudaFree avg	246 μs	3,752 μs (15×)	115,506 μs (115ms)
Bimodal peaks	4.2 μs	4.2 $\leftrightarrow$ 16.8 μs	—
Gap $\leftrightarrow$ cudaFree overlap	—	60-82%	—

그럼 MPS는 cudaFree wait을 우회할 수 있나?

MPS — Recovers Compute AI, Catastrophic for Memory AI

Same trap, different trigger

- Compute AI: MPS **RECOVERS** — NeuralRx 389→40ms, forecaster 381→42ms, qwen 185→43ms
- Memory AI: MPS **CATASTROPHIC** — sat_hbm 426 → 6,985ms (bistable 7-sec freeze)
- Why recover? True concurrent exec → compute parallelizes → cudaFree returns fast
- Why collapse? Memory hog **starves DRAM** → device truly stuck → cudaFree blocks 115ms → 7s
- No universal escape — same mechanism, different trigger



Workload	Default	MPS	MIG cross-part
NeuralRx	389ms	40ms	197ms
forecaster	381ms	42ms	—
qwen	185ms	43ms	—
sat_hbm	426ms	6,985ms	—

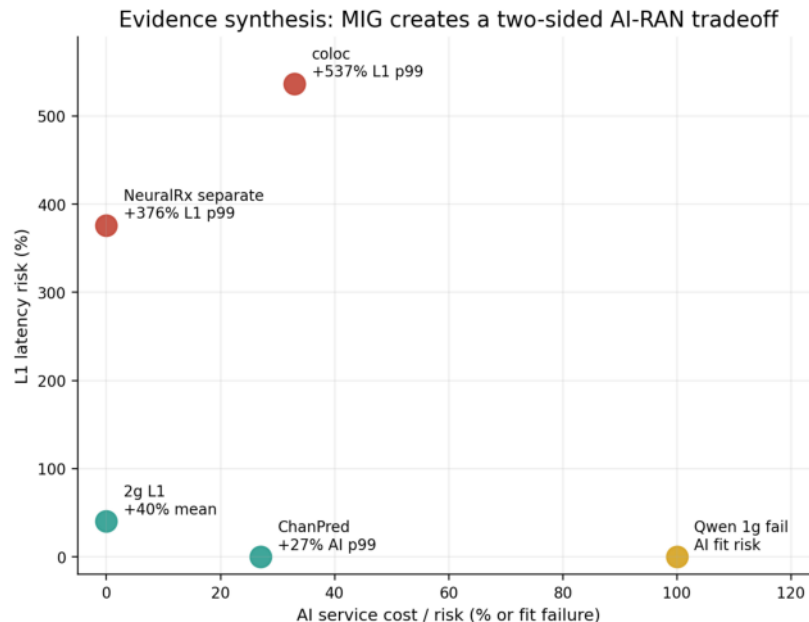
이 모든 발견을 어떻게 하나의 framework로 종합하나?

PART F §7-§8 (Perlmutter MPS 5-workload comparison)

Cost Decomposition — 3-Platform Unified Framework

Two hardware-level costs, not one — the paper contribution

- MIG overhead = TWO hardware-level costs, not one monolithic 'overhead'
- Structural (memset) = partition HBM bandwidth share — controllable by partition choice
- Contention (memcpy) = chip-wide PCIe/DMA queue — NOT controllable
- 3 platforms (MIG / no-MIG / MPS) each avoid one cost but fail at the other
- No universal isolation exists in current NVIDIA stack



Cost type	Hardware	MIG isolates?
Structural	HBM bandwidth	Yes
Contention	PCIe/DMA copy engine	NO

이 framework의 운영 함의(design rules)는?

Design Rules & Next Steps

Measurement-validated operational guidance

- Rule 1: L1과 AI는 반드시 다른 MIG partition (single mistake → 8× collapse)
- Rule 2: Partition 크기는 L1 SM/HBM headroom으로만 결정 (coloc 회피용 무의미)
- Rule 3: AI workload classification 필요 (memory-bound AI는 MPS에도 위험)
- Rule 4: MIG isolation ≠ chip-wide resource isolation (paper main contribution)
- Next: CloudLab Phase 4 (MIG-off + MPS cross-check), N-AI scaling law, driver 550 baseline

Next Step	Target
CloudLab Phase 4	MIG off + MPS — cross-check Perlmutter on same HW
N-AI scaling law	L1 + N AI same-partition, direct measurement
Driver 550 baseline	Reproduce on CloudLab d8545

End.

Thank You

Questions & Discussion

Changjong Kim

Bigdata and HPC Lab, SeoulTech

changjong5238@gmail.com